AI Usage Log - Sprint 2

Project Title: Career Connect - Job Search and Application Tracking Platform

TEMPLATE:

Team Member Name: [Insert Name]

Instructions: Every use of AI must be documented using the format below. At the end of the sprint, export this document as a PDF and upload it to your personal subdirectory within the AI_Log folder.

- **Task ID/Title:** [Insert task description]
- **Purpose of AI Use:** [Insert purpose]
- **Chat Link or Prompt/Response:** [Insert link or note appendix reference]
- **AI-Suggested Content:** [Insert summary]
- **Validation:** [Insert validation method]
- **Decision:** [Accepted / Modified before use / Rejected]
- **Reflection:** [Insert reflection]
- **Responsible Person:** [Your Name]

JULIA

Entry 1 (10/21/2026)

Team Member Name: Julia

- **Task ID/Title:** Task 4.1 - 4.5: How to Implement Job Seeker Profile Creation (MongoDB/React)
- **Purpose of AI Use:** Boilerplate creation. I used the AI to generate the initial skeleton template with TODO comments for the Mongoose schema, Express backend routes, React frontend form, and unit tests.
- **AI-Suggested Content:** The AI initially provided template code for SQL, then adapted to MongoDB/Mongoose upon correction. It generated templates for Profile.js (Mongoose schema with embedded experience/education), profile.js (Express POST route with server-side validation), CreateProfile.jsx (React form), and profile.test.js (Jest unit tests). It provided TODO markers for manual implementation.
- **Validation:** Manual review of the generated skeleton code to ensure it meets the requirements for AT-04.1, AT-04.2, and AT-04.3, and checking that the MongoDB/Mongoose approach aligns with the team's tech stack.
- **Decision:** Accepted / Modified before use (I will be filling in the TODOs manually).
- **Reflection:** The AI was very helpful in setting up the foundational structure of the files. Clarifying the database stack (SQL vs MongoDB) was necessary, but once corrected, the skeleton templates provided a clear roadmap for what needs to be implemented without writing the entire logic for me.
- **Responsible Person:** Julia

Entry 2 (10/22/2026)

Team Member Name: Julia

- **Task ID/Title:** US-05: Edit My Profile - Date Selection Widget Implementation
- **Purpose of AI Use:** Coding assistance and debugging UI accessibility
- **Chat Link or Prompt/Response:** "How do I implement a date selection widget for experience and education?", "My code isn't working and my tests are failing because the labels aren't linking to the start and end dates."
- **AI-Suggested Content:** The AI suggested templates for structuring HTML5 <input type="date"> fields and provided guidance on how to dynamically generate unique id and

htmlFor attributes within a .map() function. It also suggested the conditional logic to disable the End Date input when the "I still work here" checkbox is toggled.

- **Validation:** Manually tested the date widgets in the browser to ensure state updated correctly, and verified that the React Testing Library accessibility checks passed without DOM errors.
- **Decision:** Modified before use
- **Reflection:** I struggled to get the dynamic form inputs to link properly in my array map. The AI's suggestions helped me fix my broken code by demonstrating how to handle dynamic accessibility attributes and conditional state changes effectively.
- **Responsible Person:** Julia

Entry 3 (10/22/2026)

Team Member Name: Julia

- **Task ID/Title:** Task 4.1 - 4.5: How to Implement Job Seeker Profile Creation - Bug Fixes
- **Purpose of AI Use:** Debugging and code generation. I used the AI to troubleshoot and fix issues with date validation (end date before start date), displaying the end date, and month offset issues when saving dates in the profile forms.
- **AI-Suggested Content:** The AI provided fixes for of Profile.js (Mongoose schema), EditProfile.jsx, and Profile.jsx. It suggested a validation check to block saving if the end date precedes the start date, updated the Profile.jsx to explicitly render the end date, and created a formatDate helper function using getUTCFullYear(), getUTCMonth(), and getUTCDate() to fix the timezone offset issue.
- **Validation:** Manual review of the code and testing the implementation in the application to ensure the bugs were resolved and the date logic worked as expected.
- **Decision:** Accepted.
- **Reflection:** The AI provided pointers to fix a tricky timezone-related date offset issue that is common in JavaScript, and efficiently handled the timeline validation. It saved a significant amount of time in debugging, pointing me to the right solution.
- **Responsible Person:** Julia

Entry 4 (10/22/2026)

Team Member Name: Julia

- **Task ID/Title:** Task 4.5: How to write unit tests for the profile endpoint and validation

- **Purpose of AI Use:** Test case generation. I asked the AI pointers on what unit tests to make for the newly implemented profile endpoints and validation logic using Jest and Supertest.
- **AI-Suggested Content:** The AI suggested to make tests covering profile creation/updating, phone/URL format validation, and authentication checks. It also provided instructions on how to modify server.js to conditionally connect to the database and export the app for testing, preventing port conflict, as I am not familiar with this workflow.
- **Validation:** Manual review of the tests to ensure they cover the specified acceptance criteria (AT-04.1, AT-04.3) and running the tests using npm test.
- **Decision:** Accepted.
- **Reflection:** The AI not only provided the suggestions for possible test cases but correctly identified that modifications to server.js were required to run Supertest properly without hanging the server, which was a crucial architectural insight.
- **Responsible Person:** Julia

Entry 5 (10/22/2026)

Team Member Name: Julia

- **Task ID/Title:** Task 4.5: How to write comprehensive unit tests
- **Purpose of AI Use:** Expanding the test suite. I asked the AI to generate a full suite of tests covering all profile features developed for the current GitHub task.
- **AI-Suggested Content:** The AI provided an extensive profile.test.js file covering profile retrieval, creation/updating with optional fields, format validations (phone, URL), experience/education timelines (including the 'current' checkbox), resume uploading/downloading, and authentication security. It also recommended step-by-step instructions and a text-based README template for setting up and running the tests.
- **Validation:** Reviewing the generated test cases against all implemented features and executing them to confirm coverage and correctness.
- **Decision:** Accepted.
- **Reflection:** The comprehensive test suite generated by the AI ensured all edge cases and features were covered. The addition of the testing guide was highly useful for team documentation.
- **Responsible Person:** Julia

Entry 6: Date Selection Widget Implementation

Team Member Name: Julia

- **Task ID/Title:** US-05: Edit My Profile - Date Selection Widget
- **Purpose of AI Use:** UI component templates and troubleshooting React state
- **Chat Link or Prompt/Response:** "How do I implement a date selection widget for experience and education dates?" / "The start and end dates aren't updating correctly in my state array, and the UI keeps breaking."
- **AI-Suggested Content:** The AI provided a foundational boilerplate for HTML5 <input type="date"> fields integrated within a React .map() function for the experience/education arrays. When my initial code wouldn't work, it suggested iterative improvements, like assigning dynamic ids to the inputs and adding conditional logic to disable the end date when the "current" checkbox is selected.
- **Validation:** Manually tested the date inputs in the browser to ensure the React state updated the correct array index and that timezone shifting bugs were avoided.
- **Decision:** Modified before use
- **Reflection:** The AI's templates helped me understand how to handle complex nested state inside React arrays. It successfully guided me through fixing bugs where the wrong date was being updated in the UI, improving my understanding of controlled components and dynamic form generation.
- **Responsible Person:** Julia

Michel Yosufov

Entry 11

Task ID/Title: Build Feature 4: Recruiter Dashboard and Posting Management (US-16 to US-19, issues #28 to #31) (models/Job.js, models/Application.js, routes/jobRoutes.js, RecruiterDashboard.jsx)

Purpose of AI Use: Sprint 2 “Recruiter Dashboard and Posting Management” feature: create a job posting, edit/close/delete postings, manage postings from the dashboard, and view applications for a posting

Chat Link or Prompt/Response: https://claude.ai/code/session_019obdut6J7ggJhg2X6z7mRR

AI-Suggested Content: I used AI to help generate the Job and Application models, job routes, and dashboard page. I provided the existing project structure, including my authentication middleware, the role field in the User model, and the routing setup, so the generated code would fit our application.

Validation: I integrated the generated files into our existing server and client, then reviewed the code to make sure the routes, authentication, and token handling matched our project. I also tested each available action in the browser using a recruiter account and checked that the feature worked with our existing setup.

Decision: Accepted with modifications and validation.

Reflection: I used the AI-generated code as a starting point rather than treating it as a finished solution. I had to connect it to the authentication and role system I built in Sprint 1 and verify that it worked with our existing login. Testing with a recruiter account helped me confirm the current functionality, although I could not test the applicant workflow yet because the apply feature has not been implemented. I will retest that part once it becomes available. Before the demo, I also plan to review the routes and understand how each one works so I can explain the implementation and answer questions about it.

Responsible Person: Michel Yosufov

Entry 12

Task ID/Title: Git help: rejected push, interrupted merge, and Windows folder-lock error

Purpose of AI Use: Push the feature branches and merge main into my Sprint 2 branch without losing work

Chat Link or Prompt/Response: https://claude.ai/code/session_019obdut6J7ggJhg2X6z7mRR

AI-Suggested Content: Explained why the push was rejected (the remote had commits I lacked), how to pull and finish an unfinished merge (git status, then git commit), and that the 'Deletion of directory failed' message was only Windows locking a folder after teammates renamed files in main.

Validation: Ran git status after each step, finished the merge commit, restarted the app to confirm login and the recruiter dashboard still worked, then pushed the branch successfully.

Decision: Accepted

Reflection: Checking git status before each step made the merge understandable. Next time I will merge main before starting a feature branch to avoid a late merge.

Responsible Person: Michel Yosufov

Entry 13

Task ID/Title: Diagnose MongoDB Atlas connection crash (Could not connect to any servers in your MongoDB Atlas cluster)

Purpose of AI Use: Fix the backend crashing on startup so the project could be run and demoed

Chat Link or Prompt/Response: https://claude.ai/code/session_019obdut6J7ggJhg2X6z7mRR

AI-Suggested Content: Identified that the error came from Atlas's Network Access IP allowlist (my current IP was not on the list) and walked me through adding an entry under Network Access. For team development, suggested allowing 0.0.0.0/0 and noted the trade-off.

Validation: Added the 0.0.0.0/0 entry in Atlas, restarted the server with 'npm run dev', and confirmed the terminal printed 'Server running on port 5000' and 'MongoDB connected'.

Decision: Accepted

Reflection: The IP allowlist is a second layer on top of the password. Allowing all IPs is only acceptable for development, so the .env file must stay private (it is in .gitignore) and the 0.0.0.0/0 entry should be removed after the course.

Responsible Person: Michel Yosufov

Entry 14

Task ID/Title: Write teammate run-guide section for the README (Running CareerConnect Locally)

Purpose of AI Use: Let teammates and the TA install and run the client and server on another computer

Chat Link or Prompt/Response: https://claude.ai/code/session_019obdut6J7ggJhg2X6z7mRR

AI-Suggested Content: At my request, wrote a first-person README section covering installing Node and Git, cloning the repo, npm install in server and client, creating the .env file, the Atlas IP allowlist, starting both servers, trying the main pages, and a troubleshooting table. I supplied the project facts (repo, stack, how the .env is shared); the AI wrote the text.

Validation: Read through the steps and checked them against our project setup. It has not yet been tested by a teammate on another computer. The .env values are shared privately and never put in the repo.

Decision: Accepted

Reflection: The AI wrote gave bullet points processes which I then used to write the final step by step process of how to run the code on any computer. I should have a teammate follow it on a clean computer to confirm every step works and update it if they hit problems.

Responsible Person: Michel Yosufov

Entry 15

Task ID/Title: Understand the existing job search test file (jobs.test.js) and why automated tests matter

Purpose of AI Use: Learn what the test class does and how automated tests help a developer

Chat Link or Prompt/Response: https://claude.ai/code/session_019obdut6J7ggJhg2X6z7mRR

AI-Suggested Content: Explained the setup and cleanup steps, the four groups of tests (US-08 to US-11), and the benefits of automated tests: catching bugs early, refactoring safely, acting as documentation and proving requirements are met.

Validation: Compared the explanation with the test file and the project routes.

Decision: Accepted

Reflection: The explanation helped me read the existing test style, which I reused when writing my own tests. I still needed to read the code myself to confirm the details.

Responsible Person: Michel Yosufov

Entry 16

Task ID/Title: Plan testing for the recruiter dashboard (US-16 to US-19) with a same-day deadline

Purpose of AI Use: Decide what needed automated tests and what manual testing could cover

Chat Link or Prompt/Response: https://claude.ai/code/session_019obdut6J7ggJhg2X6z7mRR

AI-Suggested Content: Recommended automated tests for the backend API (creating, editing and deleting postings, permission checks with 401 and 403 responses) and manual testing for the user interface. For the deadline, it suggested a minimum plan of a manual checklist plus a few key permission tests.

Validation: Checked the advice against the Sprint 2 requirement of at least 2 acceptance tests per user story.

Decision: Modified before use

Reflection: I went further than the minimum plan and wrote the full set of automated tests. Manual testing only proves a feature worked once, and automated permission tests catch problems I would not find by clicking around as a single user.

Responsible Person: Michel Yosufov

Entry 17

Task ID/Title: Clarify my Sprint 2 assignments (#6, #7, #28 to #31 and “Implement Backend API Integration”)

Purpose of AI Use: List my assigned tasks from the Sprint 2 Work Plan and understand why Sprint 1 stories were included again

Chat Link or Prompt/Response: https://claude.ai/code/session_019obdut6J7ggJhg2X6z7mRR

AI-Suggested Content: Listed my user stories, development tasks and shared tasks. Explained that #6 and #7 needed acceptance tests rather than a rebuild. Gave a best-guess meaning of “Backend API Integration” and said it was not defined in the plan.

Validation: Checked the list against the Sprint 2 Work Plan document.

Decision: Accepted

Reflection: This showed me I had not yet written acceptance tests for #6 and #7. I should have read the work plan more carefully earlier in the sprint.

Responsible Person: Michel Yosufov

Entry 18

Task ID/Title: Automated acceptance tests for US-01 (Register) and US-02 (Login/Logout) and a fix to the register route

Purpose of AI Use: Write automated tests for #6 and #7 and review my authentication code

Chat Link or Prompt/Response: https://claude.ai/code/session_019obdut6J7ggJhg2X6z7mRR

AI-Suggested Content: After I uploaded the project, the AI wrote Auth.test.js covering registering with valid details, a duplicate email, a missing field, login, a wrong password, and requests with no, invalid or expired tokens. It also changed authRoutes.js so a missing name, email or password returns a 400 error instead of a 500 error.

Validation: The AI only checked the syntax. I ran npm test and all suites passed, and I tested registration, login and logout manually in the running app.

Decision: Accepted

Reflection: The AI found a real bug, where a mistake by the user caused a server error. The tests also give evidence for the acceptance-test requirement.

Responsible Person: Michel Yosufov

Entry 19

Task ID/Title: Password rule gap against the US-01 and US-02 acceptance criteria

Purpose of AI Use: Compare my authentication code with the acceptance criteria I copied from issues #6 and #7

Chat Link or Prompt/Response: https://claude.ai/code/session_019obdut6J7ggJhg2X6z7mRR

AI-Suggested Content: Found that the rule of at least 8 characters and one number was not enforced anywhere. Provided an updated authRoutes.js with password validation and a new weak-

password test. Also told me to check by hand whether protected pages redirect to login when logged out, and how logout works.

Validation: Ran npm test again and everything passed. Pushed the change to the same pull request and tested weak and valid passwords in the app.

Decision: Accepted

Reflection: Passing tests did not mean the acceptance criteria were met. I should compare the code with the criteria first, before writing tests.

Responsible Person: Michel Yosufov

Entry 20

Task ID/Title: Automated tests for the recruiter dashboard (US-16 to US-19)

Purpose of AI Use: Write tests for the routes I built in Sprint 2

Chat Link or Prompt/Response: https://claude.ai/code/session_019obdut6J7ggJhg2X6z7mRR

AI-Suggested Content: Wrote recruiterJobs.test.js, which covers creating, editing, closing and deleting postings, recruiter-only and ownership checks, the dashboard listing with application counts, and viewing applications for a posting.

Validation: The AI had no access to my database, so it could only check the syntax. I ran npm test and all tests passed, then pushed the file to my pull request.

Decision: Accepted

Reflection: The tests confirmed that my permission checks work, such as one recruiter not being able to edit another recruiter's posting. Running them against the real database was my own validation.

Responsible Person: Michel Yosufov

Entry 21

Task ID/Title: Gap check of the recruiter acceptance criteria (#28 to #31) against my implementation

Purpose of AI Use: Compare the criteria in the user story issues with the features I built

Chat Link or Prompt/Response: https://claude.ai/code/session_019obdut6J7ggJhg2X6z7mRR

AI-Suggested Content: Confirmed that AT-16.1, AT-16.2, AT-17.2, AT-17.3 and AT-18.1 match my code. Flagged AT-16.3 (past deadline message), AT-17.1 (edit the salary) and AT-18.2 (Create posting button on an empty dashboard) as not implemented. It first said the applicants list had no CV link, then corrected itself after reading my dashboard code: the link exists, but the application status is not shown.

Validation: I checked the flagged items against RecruiterDashboard.jsx and the running app.

Decision: Accepted

Reflection: Some criteria describe features that are not built yet, so I recorded them honestly as Fail or Partial and left those issues open for Sprint 3. The AI made an error on the CV link, so I should check its claims against the code myself.

Responsible Person: Michel Yosufov

Entry 22

Task ID/Title: Acceptance-test issue texts for #6, #7 and #28 to #31, and Git pull request steps

Purpose of AI Use: Draft the GitHub issues for the acceptance tests and get the branch, commit and push commands for the team workflow

Chat Link or Prompt/Response: https://claude.ai/code/session_019obdut6J7ggJhg2X6z7mRR

AI-Suggested Content: Issue templates with steps, expected result, actual result, status and date, using the IDs from the acceptance criteria in each story, including Fail and Partial versions for the features not implemented. Also the Git commands for a feature branch and a pull request into develop.

Validation: I compared each issue with my team's story criteria and with what I saw in the running app, then created the issues, and closed the ones that passed.

Decision: Modified before use

Reflection: The AI first guessed that teammates had made acceptance-test issues and invented IDs before seeing the real criteria, so I had to correct it and use the IDs from the stories. I should show the AI the real requirements earlier.

Responsible Person: Michel Yosufov

Yassen

AI Usage 1 — GitHub Workflow Clarification

Task ID/Title: GitHub Project Management

Purpose of AI Use: Clarify how GitHub Issues, labels, branches, and the Project Board could be used together for an Agile project.

Chat Link or Prompt/Response: ChatGPT conversation regarding GitHub workflow and project management.

AI-Suggested Content: Provided explanations and examples of how user stories could be represented as Issues and tracked through the Project Board.

Validation: Suggestions were compared with GitHub's available features and the project requirements before implementation.

Decision: Modified before use.

Reflection: AI was useful for clarifying unfamiliar GitHub features. The final repository structure, task organization, and workflow were configured based on the team's needs.

Responsible Person: Yassen Hegazy

AI Usage 2 — Team Process Brainstorming

Task ID/Title: Defining Team Processes

Purpose of AI Use: Brainstorm important topics that should be considered when establishing the team's development process.

Chat Link or Prompt/Response: ChatGPT conversation regarding Agile team processes.

AI-Suggested Content: Suggested considering areas such as branching, code review, task tracking, communication, and definitions of ready and done.

Validation: Suggestions were reviewed and selected based on their relevance to the team's actual workflow.

Decision: Modified before use.

Reflection: AI was mainly used as a brainstorming tool to ensure important process considerations were not overlooked. The final processes were selected and organized according to the team's workflow.

Responsible Person: Yassen Hegazy

AI Usage 3 — Technical Troubleshooting

Task ID/Title: Development Environment Troubleshooting

Purpose of AI Use: Assist with understanding and troubleshooting unfamiliar technologies encountered during initial project setup.

Chat Link or Prompt/Response: ChatGPT conversation regarding development tools and configuration.

AI-Suggested Content: Provided explanations and troubleshooting suggestions related to project dependencies, environment configuration, and database connectivity.

Validation: Suggested solutions were tested in the local development environment to determine whether they resolved the encountered issues.

Decision: Modified before use.

Reflection: AI was useful as a troubleshooting resource when encountering unfamiliar tools. Suggested solutions still required testing and adjustment within the actual project environment.

Responsible Person: Yassen Hegazy

AI Usage 4 — Sprint Documentation Review

Task ID/Title: Sprint 1 Documentation

Purpose of AI Use: Review the organization and clarity of Sprint 1 documentation and suggest improvements where necessary.

Chat Link or Prompt/Response: ChatGPT conversation regarding Sprint 1 documentation.

AI-Suggested Content: Suggested ways to organize sprint information and improve the clarity of several documentation sections.

Validation: Suggestions were compared against the actual work completed during Sprint 1 and edited to ensure the documentation accurately represented the team's activities.

Decision: Modified before use.

Reflection: AI was useful for reviewing the presentation and organization of information. Final content was adjusted to reflect the team's actual work and project decisions.

Responsible Person: Yassen Hegazy

AI Usage 5 — Multiple CV Management Implementation

Team Member Name: Yassen

- **Task ID/Title:** US-07: Keep Several CVs and Choose a Default
- **Purpose of AI Use:** Boilerplate generation and debugging backend/frontend integration
- **Chat Link or Prompt/Response:** "Can you provide a boilerplate for managing an array of CVs in MongoDB and toggling a default?" / "My code isn't working when I try to update the default CV, it keeps saving incorrectly."
- **AI-Suggested Content:** The AI provided a skeleton template for updating the Mongoose User schema to hold an array of resumes rather than a single string. It also suggested logic templates for a PATCH route to enforce the "one-default" rule, and offered iterative troubleshooting steps to fix bugs when the React UI state didn't update properly in Profile.jsx.
- **Validation:** Manually reviewed the Mongoose schema changes and extensively tested the frontend UI to ensure only one CV is set to default at any given time and file replacements work.
- **Decision:** Modified before use
- **Reflection**: Using AI to generate the initial boilerplate helped structure the backend logic for array manipulation much faster. Furthermore, the AI was a great sounding board for troubleshooting why the default toggle wasn't saving correctly, helping me understand how to properly iterate through the array and update the database state safely.
- **Responsible Person:** Yassen

AI Usage 6 — API Testing for Multiple CVs

Team Member Name: Yassen

- **Task ID/Title: US-21:** Create API test cases for backend validation (US-07)
- **Purpose of AI Use:** Test suite boilerplate and debugging API endpoints
- **Chat Link or Prompt/Response:** "Help me set up an API test boilerplate for the multiple CVs feature using Jest and Supertest." / "How do I write a test for the fallback logic when a default CV is deleted?"
- **AI-Suggested Content:** The AI provided a structural skeleton for profile.test.js using Supertest. It suggested specific scenarios that needed testing, such as verifying the 400 error when no file is attached, validating the one-default rule (PATCH), and checking the automatic fallback logic when the active CV is removed (DELETE).
- **Validation:** Ran the test suite locally using npm test to ensure all API endpoints behaved correctly, the single-default rules were enforced, and all edge cases were covered without hanging the server.
- **Decision:** Modified before use

- **Reflection:** The AI-provided boilerplate saved a lot of time in setting up the Jest/Supertest environment. Its suggestions for edge-case testing helped me write robust tests that I wouldn't have immediately thought of on my own, ensuring the deletion fallback logic was fully validated.
- **Responsible Person:** Yassen

AI Usage 7 — Tests

Team Member Name: Yassen

- **Task ID/Title:** US-22: How to create UI test cases for frontend validation and testing documentation.
- **Purpose of AI Use:** Test case boilerplate generation, testing environment setup steps, and documentation writing.
- **AI-Suggested Content:** The AI generated a comprehensive Jest and React Testing Library skeleton file (ui.test.jsx) to validate the frontend UI. The test suite included mock API calls and covered user stories from Sprint 1 and Sprint 2, specifically US-01 through US-07. Additionally, the AI helped generate a frontend testing guide (README.md) detailing how to install the required testing dependencies and execute the automated tests, helping not only me to run them, but also teammates.
- **Validation:** Manually reviewed the generated test code and executed the test suite locally using the npm test command provided in the generated documentation.
- **Decision:** Accepted and modified before use.
- **Reflection:** Using AI provided a robust foundational skeleton for testing React components and saved significant time setting up the Jest/jsdom environment. It also offered valuable troubleshooting steps for configuring Babel to work with Vite and fixing HTML5 form validation issues that surfaced during testing. I now understand how test generation works.
- **Responsible Person:** Yassen

AI Usage 8 — Test debugging

Team Member Name: Yassen

- **Task ID/Title:** US-22: Fix UI Test Accessibility Error

- **Purpose of AI Use:** Debugging and problem solving
- **Chat Link or Prompt/Response:** Prompted AI with the Jest error log containing TestingLibraryElementError: Found a label with the text of: /Start Date:/i, however no form control was found associated to that label.
- **AI-Suggested Content:** Suggested updating the EditProfile.jsx component by adding htmlFor attributes to the labels and matching id attributes to the date inputs so the testing library could programmatically associate them.
- **Validation:** Testing and manual review; verified by re-running npm test to ensure the component passed the DOM accessibility checks.
- **Decision:** Accepted
- **Reflection:** Learned that React Testing Library enforces strict accessibility standards and will fail tests if form inputs are not explicitly linked to their descriptive labels. This fix worked and I could quickly move on to other tasks.
- **Responsible Person:** Yassen

AI Usage 9 — Test setup debugging

Team Member Name: Yassen

- **Task ID/Title:** US-22: Fix Jest Test Environment Setup
- **Purpose of AI Use:** Debugging and problem solving
- **Chat Link or Prompt/Response:** Prompted AI with the Jest error log containing ReferenceError: TextEncoder is not defined when attempting to mount <MemoryRouter>.
- **AI-Suggested Content:** Suggested creating a jest.setup.js file to globally define TextEncoder and TextDecoder from Node's util package, and configuring Jest to load this file before running tests.
- **Validation:** Testing; verified by re-running npm test to confirm the simulated browser environment loaded successfully without throwing reference errors.
- **Decision:** Accepted
- **Reflection:** Learned that Jest's default simulated browser environment (jsdom) does not natively include all modern web APIs (like TextEncoder) required by newer packages like React Router, requiring custom setup files.
- **Responsible Person:** Yassen

JOSE

Usage 1 : Bug fixes

- **Task ID/Title:** Full-Stack Profile Management, File Upload/Download, and UI Layout Refinement
- **Purpose of AI Use:** To troubleshoot routing bugs, implement secure file download handling, set up Git ignore rules, and resolve form CSS width and scrolling layout issues.
- **Chat Link or Prompt/Response:** https://gemini.google.com/app/8ae0e797c835130d
- **AI-Suggested Content:** Provided implementations for React Router's useNavigate, frontend blob download logic, Gitignore rules with .gitkeep, uniform CSS form box styles, text wrapping rules, and a scrollable container for long headlines.
- **Validation:** Visual verification via screenshots and functional testing of application routes and code snippets.
- **Decision:** Modified before use
- **Reflection:** Iterative debugging successfully resolved component width mismatches and provided a clean, professional user experience for viewing and editing profile details.
- **Responsible Person:** Jose David Torres

Usage 2: Job Application Feature

- **Task ID/Title:** Sprint 2: US-12 - Job Application Feature with CV Selection and Cover Letter Upload
- **Purpose of AI Use:** Assisted with full-stack MERN debugging (frontend FormData migration, Express/Multer middleware configuration, Mongoose strict mode schema updates, route order resolution) and writing automated unit/UI tests (Jest, Supertest, and React Testing Library).
- **Chat Link or Prompt/Response:** https://gemini.google.com/app/8ae0e797c835130d
- **AI-Suggested Content:**
    - Refactored JobDetails.jsx frontend submit handler to use FormData without manual Content-Type headers for multipart file boundaries.
    - Added Multer middleware (upload.single('coverLetterFile')) and corrected route ordering in backend jobRoutes.js.
    - Updated the Mongoose Application schema to include coverLetterFilename and status fields with a unique composite index on (job, applicant).

    - Created backend tests in server/tests/applications.test.js and updated frontend UI tests in JobDetails.test.jsx wrapped with MemoryRouter and Routes.
- **Validation:** Verified via manual workflow checks and automated test suites ensuring correct status codes (201, 409, 404), duplicate prevention, and missing CV modal flows.
- **Decision:** Modified before use.
- **Reflection:** AI collaboration significantly accelerated troubleshooting multipart data transmission over HTTP, diagnosing Mongoose schema strict-mode exclusions, and structuring unit tests to mirror existing codebase patterns.
- **Responsible Person:** Jose David Torres

Usage 3 : Application Status

- **Task ID/Title:** Sprint 2: US-13 - Application Confirmation and Tracking Feature & Acceptance Criteria Verification (AT-12.1, AT-13.1, AT-13.2)
- **Purpose of AI Use:** Assisted with full-stack implementation of inline success confirmation screens, persistent error states retaining form/cover letter data, backend job title population on applications, and debugging/synchronizing Jest UI and Supertest integration tests.
- **Chat Link or Prompt/Response:** https://gemini.google.com/app/8ae0e797c835130d
- **AI-Suggested Content:**
- Refactored JobDetails.jsx frontend component to replace alert() feedback with an inline success confirmation view (displaying job title, date, and a link to /applications) and an inline error display that preserves the CV picker modal state on failure.
- Updated backend POST /:id/apply route to populate and return jobTitle, createdAt, status, and resumeFilename.
- Added backend test coverage and synchronized frontend UI tests in JobDetails.test.jsx with proper mock sequencing for profile fetching, duplicate prevention (409 conflict checks), and text ambiguity handling.
- **Validation:** Verified via automated Supertest integration suites and React Testing Library UI tests, ensuring 100% passing results across all user story acceptance criteria (AT-12.1, AT-13.1, and AT-13.2).
- **Decision:** Accepted and integrated directly into the codebase.
- **Reflection:** AI collaboration streamlined full-stack synchronization—particularly resolving asynchronous mock fetch sequencing, Mongoose population constraints, and Testing Library query scoping for multi-element text matches.
- **Responsible Person:** Jose David Torres

NOOR

Task ID/Title: US-01-->US-19 — User Stories and Task Breakdown

Purpose of AI Use: Understanding user story structure and brainstorming requirements

Chat Link or Prompt/Response:
Prompt: "How should a user story be written, what should its acceptance criteria include, and can you give me a few examples for a software project?"
Response: AI explained the general structure of a user story, acceptance criteria, and tasks, with examples related to common software features.

AI-Suggested Content: General examples and ideas for structuring user stories, acceptance criteria, and tasks.

Validation: Compared the ideas with the Sprint 1 requirements, then developed and organized my own stories and tasks.

Decision: Modified before use

Reflection: The explanation helped me understand how to structure the stories and think about possible requirements. I used it mainly for brainstorming and then wrote the final acceptance criteria and tasks myself.

Responsible Person: Noor Rabie

Task ID/Title: —Git Workflow and Debugging

Purpose of AI Use: : Debugging Git and build errors

Chat Link or Prompt/Response:

Prompt: Prompt: "git pull gives an error / how do I go back to develop / I opened my PR into main instead of develop, how do I fix it?"

Response: AI explained the cause of each error and how to fix it, and explained our feature-branch → develop workflow.

AI-Suggested Content: Explanations and commands to fix branch and PR issues.

Validation: Ran the commands myself and checked the results on GitHub and against our Team_Process.md.

Decision: Accepted

Reflection: I understand branches and PRs much better now, and I always check the base branch before opening a PR.

Responsible Person: Noor Rabie

Task ID/Title: — Debugging and Fixing a Code Error

Purpose of AI Use: Identifying the cause of an error in the code and understanding how to fix it

Chat Link or Prompt/Response:

Prompt: “I’m getting this error in this part of my code. What could be causing it and how can I fix it?”

Response: AI explained what the error message meant, pointed out the likely cause in the code (e.g., [briefly describe: wrong variable type, missing null check, incorrect function call, etc.]), and suggested possible ways to fix it.

AI-Suggested Content: An explanation of the error, the possible cause, and a suggested fix.

Validation: Reviewed the suggested cause against my code, applied the fix, and ran the program and tests to confirm the error was resolved and nothing else broke.

Decision: Modified before use

Reflection: The response helped me understand why the error happened, not just how to remove it. I adjusted the suggested fix to fit our code structure and tested it myself before committing the change.

Responsible Person: Noor Rabie

Task ID/Title: US-08 → US-10 — Fixing a File Name Error Before Merging

Purpose of AI Use: Debugging

Chat Link or Prompt/Response: (screenshot of the VS Code error) "Already included file name 'Joblist.jsx' differs from file name 'joblist.jsx' only in casing."

Response: AI explained that my file name and my import used different capital letters. Windows ignores this, so it worked on my computer, but our CI pipeline runs on Linux, where it would make the build fail.

AI-Suggested Content: Rename the file and update the import so both match exactly, then reload VS Code.

Validation: Renamed the file, confirmed Git recorded the correct name, ran lint and build locally, and checked that the CI pipeline passed after merging.

Decision: Accepted

Reflection: I learned that code can work on my machine but fail on the server because of differences between operating systems, and that running lint and build before pushing catches these problems early.

Responsible Person: Noor Rabie

Task ID/Title: US-08 → US-11 — Fixing Failing Backend Tests

Purpose of AI Use: Debugging and testing

Chat Link or Prompt/Response: "My job search tests are failing with 'Exceeded timeout of 5000 ms for a hook', what's wrong?"

Response: AI explained that the tests connect to our shared cloud database, which can be slow or briefly unreachable, so the setup took longer than Jest's 5-second default. It also pointed out that if setup failed halfway, the cleanup step would crash and could leave test data in the team's database.

AI-Suggested Content: Increase the test timeout, and make the cleanup find test data by its unique names so it still works if setup fails.

Validation: Re-ran the tests until all passed, checked the database to confirm no leftover test data, and confirmed the CI pipeline passed after merging.

Decision: Modified before use

Reflection: I learned that tests using a shared database need to clean up after themselves carefully, so they don't affect teammates' data, and that a failing test isn't always a code bug; it can be the environment.

Responsible Person: Noor Rabie